

ROBOTON — Complete Implementation & Architecture Guide

A build-it-with-me guide from the SIH 2026 (PS 26123) Phase-Wise Roadmap

Legend used throughout this document: - **PDF REQUIREMENT** — explicitly stated in the roadmap/SRS. - **IMPLEMENTATION RECOMMENDATION** — my suggestion for *how* to satisfy a PDF requirement. - **THE PDF DOES NOT SPECIFY THIS — HERE IS A RECOMMENDATION** — a gap the PDF leaves open, filled with a practical default.

1. What We Are Building

PDF REQUIREMENT: A decentralized multi-robot coordination system (ROBOTON) where the *robots themselves* carry the essential decision logic — planning, bidding, intent-sharing, junction reservation, and arbitration. The gateway (task injection) and dashboard (monitoring) are optional/convenience layers, not decision-makers. The graded SIH deliverable is a **simulation with ≥ 3 AMRs**, a **baseline controller**, and a **benchmark harness** proving safety (zero collisions) and performance ($\geq 20\%$ makespan reduction vs. stop-and-wait).

Think of it as: *three independent robot “brains,” each running the same code, that talk to each other over a shared message bus and negotiate space in real time — with a simulator standing in for the physical world, and a passive dashboard just watching.*

2. Software / Tools Required

A. Software we MUST build (this is the actual project)

- All of core/ — robot logic, planners, auction, reservation, arbitration, traffic model, clock, localization simulation.
- All of communication/ — the message formats and the transport layer that moves them between robot processes.
- All of simulator/ — the glue code that turns our robot logic into visible movement (this is *not* the simulator engine itself, it’s our adapter code).
- benchmark/ — baseline vs. ROBOTON comparison harness.
- dashboard/backend and dashboard/frontend — read-only visualization.

B. Software/libraries we can use (off-the-shelf, PDF-sanctioned)

- **PDF REQUIREMENT:** A simulator — **Webots or Gazebo** (roadmap explicitly names both and tells the team to pick one; it does not pick for you).
- **THE PDF DOES NOT SPECIFY THIS — HERE IS A RECOMMENDATION:** Choose **Webots**. Reasoning: Webots has a much gentler learning curve for a topological-graph warehouse (no

need to fight ROS2 launch files or URDF pipelines just to get three boxes moving between nodes), ships its own Python API out of the box, runs cross-platform without a ROS install, and is normally enough to satisfy a jury visually. Gazebo is the better choice only if your team already knows ROS2 or if later phases require tight ROS ecosystem integration (e.g., navigation stacks) — which this project explicitly does *not* need, since the routing/coordination brain is your own code, not a ROS nav stack.

- Python standard library (asyncio, socket, json, dataclasses, heapq for A*).
- A lightweight web framework for the dashboard backend (e.g., FastAPI/Flask) and a simple frontend (plain JS/React) — **not named by the PDF**, your choice.
- networkx (optional) to sanity-check your own graph implementation during development — not required, since the PDF wants you to own graph.py.

C. Software that is optional

- Dashboard entirely (PDF: “dashboard/gateway are optional convenience components” — build it, but it is not what gets you AC compliance).
- Zoning/fleet-scale simulation (Phase 13) — needed only if you want to demonstrate scaling.
- Localization/clock simulation realism (Phase 10) beyond a simple confidence counter.

D. Hardware software that comes later

- **PDF REQUIREMENT:** ESP32 firmware, ESP-NOW peer transport, motor/encoder/IMU drivers — **Phase 14 only**, after the simulated system is fully benchmarked. The PDF is explicit: “Do not let physical hardware become a single point of failure for the SIH deliverable.”

Programming language(s)

THE PDF DOES NOT SPECIFY THIS — HERE IS A RECOMMENDATION: Python for everything in simulation (core/, communication/, simulator/, benchmark/, dashboard backend). This matches your existing skill level (Python + basic ML/Flask) and Webots’ native Python controller API. C/C++ only appears later for ESP32 firmware (Phase 14), which uses the Arduino/ESP-IDF toolchain, not Python.

Backend requirements

IMPLEMENTATION RECOMMENDATION: A small backend process (FastAPI/Flask) that subscribes to TELEMETRY messages from the robot processes and exposes them over a WebSocket/HTTP endpoint for the dashboard. It must never send commands back — it only relays.

Communication/messaging requirements

PDF REQUIREMENT (exact message table):

Message	Sender → Receiver	Frequency	Purpose
ANNOUNCE	Gateway → mesh	on new task	publish task, doesn’t pick winner

BID	Robot ↔ robot	during 300 ms auction window	local task cost
CLAIM	Robot → mesh	after bid decision	declare winner so all converge
INTENT	Robot → nearby robots	every 200 ms	future path/ETA for safety
RESERVE	Robot → nearby robots	before junction entry	claim a time window
DIGEST	Robot ↔ robot	every 10 s	share learned traffic estimates
TELEMETRY	Mesh → dashboard	~1 Hz	monitoring only

Database/logging requirements

THE PDF DOES NOT SPECIFY THIS — HERE IS A

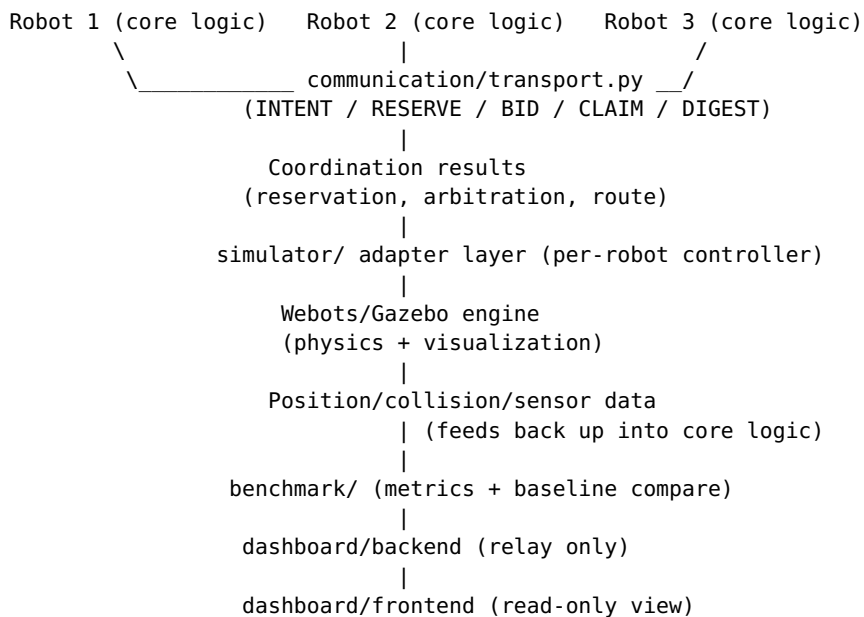
RECOMMENDATION: No database needed for the simulation. Use flat CSV/JSON log files per benchmark run (the PDF explicitly asks for “CSV/JSON logs” as the benchmark deliverable). A database only becomes useful at fleet scale (Phase 13) or hardware telemetry retention (Phase 14), and even then SQLite is enough.

Benchmarking tools

PDF REQUIREMENT: Your own benchmark/runner.py comparing Configuration A (stop-and-wait) vs Configuration B (full ROBOTON) on identical map/tasks/seeds, ≥ 10 seeds each, reporting mean + standard deviation of makespan, mean task completion time, collision count, stopped time, yields, auction frames/task, route diversity. **THE PDF DOES NOT SPECIFY THIS — HERE IS A**

RECOMMENDATION: Use matplotlib or plotly for the comparison charts — plain, no extra infra.

Accurate architecture



3. Simulator: What It Actually Does vs. What Our Code Does

Responsibility	Owned by
Physics, rendering, robot geometry, collision <i>detection</i> at the geometric level, time-stepping	Webots/Gazebo (the simulator)
Warehouse layout as a 3D/2D scene	Simulator scene , generated from our graph data
What a junction/aisle/wall <i>means</i> logically (graph nodes/edges)	Our code (core/graph.py) — simulator only sees geometry, not meaning
Sensor readings (fed by simulator)	Simulator produces raw values; our code interprets them
Deciding <i>where</i> a robot goes next	Our code (A*, auction, reservation, arbitration)
Moving the robot along that decision	Simulator , driven by velocity/waypoint commands from our controller
Collision as a <i>safety event</i> (was arbitration violated?)	Our code — the simulator only tells you two robots overlapped; our code interprets whether that means the algorithm failed
Inter-robot communication (INTENT, RESERVE, etc.)	Our code , entirely outside the simulator — simulator has no concept of this
Time/ETA	Our code computes ETA from graph costs; simulator's clock is just used to keep it synced

Concrete example (from the PDF's own message table + arbitration rule)

Robot 1 wants to travel A → B
|
A* (core/planner_astar.py) computes route over the graph
|
simulator adapter issues movement commands; Webots moves the robot mesh
|
Robot 1's INTENT (next 3 nodes + ETA) is broadcast every 200 ms
|
Robot 2's INTENT shows it will also reach the same junction within the 800 ms safety margin → conflict detected (core/reservation.py)
|
Deterministic arbitration (core/arbitration.py): priority first, robot_id breaks ties
|
Winner broadcasts RESERVE; loser yields (shifts ETA / alternate edge)
|
Simulator visually shows Robot 1 continuing, Robot 2 slowing/rerouting

Every “decision” box above is our Python code running *outside* Webots; Webots only executes the resulting motion and shows it.

4. Building the Simulation, Step by Step

This section maps 1:1 onto the roadmap's phases (I've merged your 18-step outline into the PDF's actual 15 phases — the PDF's numbering is the one to defend in your presentation).

For each phase: what you see, what you write, in/out, connection to next step, and what to test — pulled directly from the PDF's own "What the team does / Deliverable / Stop-go gate" for that phase.

Phase 0 — Freeze the MVP. *See:* nothing visual yet. *Write:* repo structure, issue board, terminology glossary (INTENT/RESERVE/ANNOUNCE/BID/CLAIM/DIGEST). *Test:* everyone agrees on what "decentralized" means before code starts.

Phase 1 — Warehouse graph. *See:* a 2D plotted map of 9–15 junctions with a chokepoint corridor and two longer alternatives. *Write:* `core/graph.py` (nodes, edges, costs, single-lane flags) + a visualizer script. *In:* node/edge definitions. *Out:* a queryable graph object. *Test:* given any start/goal, valid connected paths are returned; blocked edges are handled.

Phase 2 — One simulated robot. *See:* one robot mesh moving from node A to node B in Webots. *Write:* `core/robot.py`, `core/state_machine.py` (IDLE→BIDDING→PLANNING→MOVING→YIELD/REPLAN→IDLE), `simulator/robot_model.py`, `simulator/collision_detector.py` (added now even with one robot, per the PDF). *Test:* 20 repeated A→B runs, no state bugs.

Phase 3 — A*. *See:* the route highlighted on the map before the robot moves. *Write:* `core/planner_astar.py` + unit tests. *In:* graph + start/goal. *Out:* ordered node/edge route, with unreachable goals rejected cleanly. *Test:* known maps → expected least-cost routes.

Phase 4 — Three independent robots. *See:* three robots moving simultaneously, each planning on its own — collisions are *allowed* to happen here, on purpose (PDF: "that is intentional: you are establishing the problem the coordination layer must solve"). *Write:* `core/task.py`, spawn logic for 3 robots. *Test:* each robot completes tasks with zero central path computation.

Phase 5 — Distributed task allocation. *See:* a task announced, three robots each silently computing a bid, one CLAIMs it. *Write:* `core/auction.py`, `communication/auction_messages.py`. *In:* ANNOUNCE + each robot's current route/battery/queue. *Out:* insertion-cost bids, CLAIM message. *Test:* a busy robot passing the pickup vs. a distant idle robot — the lower insertion-cost robot should win (also test the reverse/backtracking case).

Phase 6 — INTENT + reservation + arbitration. *See:* two robots approaching a shared junction; one visibly slows/reroutes while the other proceeds. *Write:* `communication/intent.py`, `communication/reserve.py`, `core/reservation.py`, `core/arbitration.py`. *In:* INTENT broadcasts (next 3 nodes, ETA, priority, battery, pheromone, timestamp) every 200 ms. *Out:* RESERVE winner, yield instruction for loser. *Test:* repeated crossing-junction runs — lower-ranked robot always yields, zero collisions.

Phase 7 — Deadlock + single-lane proof. *See:* a 3-robot cyclic conflict resolved with exactly one robot proceeding; a single-lane corridor where one robot yields *before* entering, never a head-on standoff. *Write:* automated test scenarios (TC-1/2/3 style) under `tests/coordination/`. *Test:* identical decisions across repeated runs — no cyclic wait.

Phase 8 — Traffic learning. *See:* congest one corridor repeatedly, watch later routes shift to alternatives. *Write:* core/traffic_model.py (EWMA + pheromone), communication/digest.py. *Test:* even with desynchronized learned models across robots, safety never breaks — traffic learning stays *outside* arbitration.

Phase 9 — Replanning/exceptions. *See:* a blocked aisle triggers visible rerouting; a “killed” robot’s reservations expire and its task gets re-announced. *Write:* core/planner_dstar.py (only now, after A* + reservation are stable), exception handling in core/robot.py. *Test:* blocked edge, peer failure, gateway loss, Wi-Fi loss, low battery — each produces the documented local behavior.

Phase 10 — Localization/clock simulation. *See:* nothing new visually unless you fake noisy odometry; confidence decreasing between junction markers. *Write:* core/localization.py, core/clock.py. *Test:* a stale clock beacon (widens safety margin to 1500 ms) reduces throughput but never causes a collision.

Phase 11 — Benchmark + baseline. *See:* two side-by-side runs (stop-and-wait vs. ROBOTON) on the same map/tasks/seeds. *Write:* benchmark/baseline_stop_wait.py, benchmark/task_generator.py, benchmark/metrics.py, benchmark/runner.py. *Test:* zero collisions for ROBOTON across ≥ 10 seeds, $\geq 20\%$ makespan reduction.

Phase 12 — Dashboard. *See:* a browser page showing robot positions, battery, state, alerts, fleet health, refreshing ≥ 1 Hz. *Write:* dashboard/backend, dashboard/frontend. *Test:* disconnect the dashboard — fleet keeps operating unaffected.

Phase 13 — Zoning (optional, scale proof). *See:* the same architecture at 20/50/100 simulated robots, with auction traffic staying local to a zone. *Write:* zone partitioning in core/graph.py, zone_id field in ANNOUNCE.

Phase 14 — Physical demonstrator. Only after everything above is proven. ESP32 + ESP-NOW + differential drive, 1 robot $\rightarrow$ 2 robots $\rightarrow$ 3 robots, in that order.

5. Code Architecture — Module by Module

PDF REQUIREMENT: this exact folder layout (from the roadmap’s “Recommended code structure”). Below, for each module: purpose / data in / data out / callers / where it runs / example.

core/

- **robot.py** — Purpose: the robot object (id, position, state, battery, task, route, priority) and its main decision loop. In: sensor/comm data. Out: movement commands + outgoing messages. Called by: the simulator adapter (drives it every tick). Runs: conceptually “on the robot” — in the simulator, it’s a Python process/controller per robot. Example: robot.step() reads current INTENT table, decides to move/yield/replan.
- **state_machine.py** — Purpose: enforce IDLE $\rightarrow$ BIDDING $\rightarrow$ PLANNING $\rightarrow$ MOVING $\rightarrow$ YIELD/REPLAN $\rightarrow$ IDLE. In: events (task won, conflict detected, arrived). Out: current state. Called by: robot.py. Runs: robot process.
- **task.py** — Purpose: task data structure (task_id, pickup, drop, priority, age). In: ANNOUNCE payload. Out: task object for auction. Called by: auction.py.
- **graph.py** — Purpose: warehouse topology (nodes/edges/costs/lane properties). In: map definition file. Out: neighbor queries, path

validity, cost lookups. Called by: `planner_astar.py`,
`planner_dstar.py`, `auction.py` (insertion cost),
`simulator/warehouse.py`.

- **planner_astar.py** — Purpose: shortest-cost route computation. In: graph + start/goal. Out: ordered route. Called by: `robot.py` on PLANNING state, `auction.py` for bid estimation.
- **planner_dstar.py** — Purpose: incremental route repair when costs/blockage change (built after A* is stable, per Phase 9). In: existing route + changed edge. Out: repaired route.
- **traffic_model.py** — Purpose: per-edge EWMA travel time + decaying pheromone. In: traversal times, DIGEST updates from peers. Out: adjusted edge costs (fed into A/D as a *cost*, never into arbitration). Called by: `planner_astar.py` (cost function), `digest.py`.
- **auction.py** — Purpose: compute insertion cost for a new task, decide whether to bid, resolve CLAIM conflicts deterministically. In: ANNOUNCE, current route, battery, queue depth. Out: BID value, CLAIM. Called by: `robot.py`.
- **reservation.py** — Purpose: local table of who holds which junction/time-window, built from received INTENT/RESERVE. In: INTENT/RESERVE messages. Out: conflict flags for arbitration.py.
- **arbitration.py** — Purpose: Detect → Prioritize → Reserve → Avoid. In: conflicting reservation entries. Out: RESERVE (winner) or yield instruction (loser), using priority-then-robot_id ordering.
- **localization.py** — Purpose: simulate odometry drift + marker-reset + confidence value. In: simulated encoder/IMU ticks, marker events. Out: position estimate + confidence.
- **clock.py** — Purpose: shared fleet clock + beacon staleness handling. In: beacon events (every 5 s). Out: aligned timestamps for ETA/reservation windows; widens safety margin to 1500 ms if beacon is stale.

communication/

- **messages.py** — Purpose: shared message envelope/schema helpers. In/out: raw dicts ↔ typed message objects.
- **intent.py**, **reserve.py**, **auction_messages.py**, **digest.py** — Purpose: build/parse each specific message type listed in the message table (Section 2). Called by: `robot.py`, `auction.py`, `reservation.py`, `traffic_model.py`.
- **transport.py** — Purpose: the actual send/receive mechanism between robot processes (see Section 6 below — this is the one file that decides sockets vs. queue vs. something else). Called by: every message module above.

simulator/

- **warehouse.py** — Purpose: builds the Webots/Gazebo scene from `core/graph.py` data (walls, aisles, junction markers). Runs: simulator setup, once per run.
- **robot_model.py** — Purpose: the per-robot controller script that Webots executes each tick; bridges `core/robot.py` decisions into simulator motor commands, and simulator position/sensor data back into `core/robot.py`. This is the single most important “glue” file — see Section 6.
- **obstacles.py** — Purpose: static/dynamic obstacles (blocked aisles for Phase 9 exceptions).
- **collision_detector.py** — Purpose: geometric overlap check, logged as a safety-violation event, consumed by the benchmark’s collision-count metric.

benchmark/

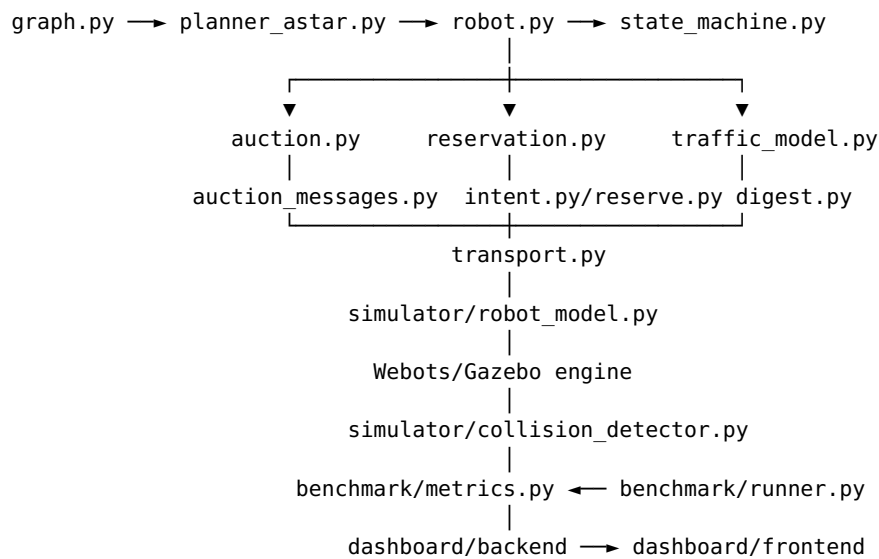
- **baseline_stop_wait.py** — Purpose: Configuration A — round-robin

- task assignment, no INTENT exchange, fixed-radius halt/resume.
- **task_generator.py** — Purpose: produce identical task sets from a fixed seed for both configurations.
- **metrics.py** — Purpose: compute makespan, mean completion time, collisions, stopped time, yields, auction frames/task, route diversity from logs.
- **runner.py** — Purpose: orchestrate N seeds × both configurations, write CSV/JSON, produce comparison report.

dashboard/backend and dashboard/frontend

- Backend: Purpose: subscribe to TELEMETRY, relay to frontend over WebSocket/HTTP; never issues commands. Frontend: Purpose: render robot positions/battery/state/alerts, read-only.

Final dependency flow



6. How Simulation Links to Our Software (the part you most need)

PDF REQUIREMENT (implicitly, by omission): the PDF never mentions ROS/ROS2 — it only names Webots/Gazebo as the simulator and ESP-NOW as the *physical hardware* transport. Do not assume ROS.

THE PDF DOES NOT SPECIFY THIS — HERE IS A RECOMMENDATION (simplest architecture that satisfies everything above):

1. **One OS-level process per robot**, running `core/robot.py`'s decision loop. This is what makes it genuinely decentralized — there is no shared memory between robots, only messages, exactly like real hardware would be.
2. **communication/transport.py** connects those processes using **plain UDP sockets (or, even simpler for a first pass, an in-process asyncio message bus if you start single-machine before going multi-process)**. This mirrors ESP-NOW's broadcast nature without needing real networking hardware, and avoids ROS2's DDS complexity entirely, which the project doesn't need since you already own the entire decision stack yourself.
3. **simulator/robot_model.py** is a Webots **controller script** — Webots launches one instance of this script per robot node in the scene.

Each instance imports and drives one `core/robot.py` object. This is the actual API boundary: Webots' own Python Robot/Supervisor API calls into your controller each simulation tick.

Downward flow (our code → simulator):

```
core/robot.py decides "move toward next_node"
|
simulator/robot_model.py translates that into Webots motor velocity
/
waypoint API calls (e.g., robot.getDevice('motor').setVelocity(...))
|
Webots physics engine moves the robot mesh
```

Upward flow (simulator → our code):

```
Webots reports position/GPS sensor value + collision proximity
sensor value
|
simulator/robot_model.py reads these via Webots' device API each
tick
|
Feeds them into core/localization.py (position/confidence) and
simulator/collision_detector.py (safety-violation events)
|
core/robot.py's state machine reacts (e.g., triggers REPLAN)
```

So: **not ROS, not sockets-to-simulator** — the simulator connection is a native controller API call (Webots' own Python bindings), which is far simpler than standing up a ROS2 node graph. The **robot-to-robot** connection (INTENT/RESERVE/BID/CLAIM/DIGEST) is the one place you do need real inter-process messaging, and plain UDP sockets are enough because message sizes are tiny and loss-tolerance is already designed into the protocol (INTENT repeats every 200 ms, so a dropped packet is self-healing).

7. Visually Demonstrating Each Algorithm

IMPLEMENTATION RECOMMENDATION for each (the PDF specifies *what* must be provable per its acceptance checklist and demo sequence; the exact visualization technique below is my recommendation):

- **A*** — Draw the full graph faintly, overlay the chosen route in a bright color, and (optionally) fade-in 1-2 alternative routes that were rejected with their higher cost labeled, so the jury sees *why* this route won.
- **Distributed auction** — A small event log panel: ANNOUNCE task_7 → BID r1=12.4 BID r2=9.1 BID r3=15.0 → CLAIM r2. Timestamp each line so the 300 ms window is visible.
- **INTENT** — Draw each robot's next-3-node path as a thin projected line ahead of it, with ETA labels at each junction.
- **RESERVATION** — At a junction node, show a small timeline bar per approaching robot indicating its reserved time window; highlight overlap before arbitration resolves it.
- **Deterministic arbitration** — Show both robots' priority + robot_id side by side, then a checkmark/cross indicating winner/yielder.
- **Deadlock handling** — Force a 3-robot cyclic scenario on a small loop map; log the same priority ordering computed independently by all three, then show exactly one proceeding.
- **Single-lane corridor** — Two robots approach from opposite ends; show the "last passing point" marker where the yield decision is forced, and confirm they never overlap inside the corridor.

- **Traffic learning** — A live-updating cost graph per edge (EWMA value + pheromone bar) next to the map; congest one corridor repeatedly and show the route line switching to an alternative once cost crosses over.
 - **Failure scenarios** — Visually mark a blocked aisle in red, show the rerouted path appear; kill a robot process and show its reservations disappear and its task re-ANNOUNCE.
-

8. Connecting Simulator → Dashboard

PDF REQUIREMENT: dashboard is passive/read-only, shows position, battery %, state, current task/progress, alerts (faults, unreachable tasks, blocked edges, low battery, collisions), fleet health (count by state, queue depth, gateway connectivity), refresh ≥ 1 Hz, must show a stale/disconnected indicator, and must be derivable from existing INTENT/relayed data rather than a separate command/query channel.

IMPLEMENTATION RECOMMENDATION for the mechanics:

What's sent: a TELEMETRY message per robot per tick — id, node/position, battery, state, current task id, last INTENT/RESERVE summary. - **How often:** ~ 1 Hz, per the PDF's own message table. - **Positions:** send graph-node + interpolated progress-along-edge (0-1), so the frontend can smoothly animate between nodes without needing raw simulator coordinates. - **Battery/task:** plain numeric/string fields in the TELEMETRY payload. - **INTENT/RESERVE visualization:** the dashboard backend simply relays the same INTENT/RESERVE messages robots already broadcast for coordination — it does not compute anything new, keeping it provably passive. - **Alerts:** derived rule-based on the backend (e.g., battery < threshold, blocked-edge event received, collision_detector event received) — never derived by asking a robot for status. - **Read-only enforcement:** simplest guarantee — the dashboard backend's socket/connection to the robot mesh is **receive-only** at the transport layer (it never has a send() path wired to transport.py's robot-facing channel). That's a structural guarantee, not just a UI rule. - **Showing decentralization visually:** render each robot with its own independent "decision panel" (current state, last bid, last arbitration result) rather than one shared "system status," so the jury visibly sees three separate brains, not one controller.

9. The Benchmark Simulation

PDF REQUIREMENT (exact protocol):

Same warehouse, same tasks, same random seeds, same robot count

```

|
Configuration A: stop-and-wait baseline
(round-robin assignment, no INTENT exchange, fixed-radius
halt/resume)

```

```

|
Collect metrics

```

Same warehouse, same tasks, same random seeds

```

|
Configuration B: full ROBOTON

```

```

|
Collect metrics

```

- Benchmark map: one single-lane choke corridor + ≥ 2 longer alternatives; $\geq 60\%$ of tasks must naturally use the choke corridor.

- ≥ 10 seeds per configuration; report **mean and standard deviation**, never a single cherry-picked run.
- **Metrics to compute** (all pulled directly from logged events):
 - **Makespan** = timestamp of last task completion – start time.
 - **Mean task completion time** = average over all tasks of (completion – announce time).
 - **Collision count** = events from `collision_detector.py`.
 - **Stopped time** = total simulated time robots spend in a halted/yielding state.
 - **Yields** = count of arbitration losses.
 - **Auction frames/task** = number of BID messages exchanged per task.
 - **Route diversity** = count of distinct edges used across all completed routes $\div$ total edges available (or similar normalized measure).
- **Success gate:** ROBOTON must show **zero** inter-robot collisions and **$\geq 20\%$ makespan reduction** vs. Configuration A.
- Tune α , ϵ , δ , w_1 , w_2 (traffic-model/auction weighting parameters) offline against makespan, then **freeze** them for the final graded run — do not keep tuning after the benchmark is “final.”

IMPLEMENTATION RECOMMENDATION for graphs: a grouped bar chart (A vs. B) for makespan/collisions/stopped-time with error bars from the standard deviation across seeds — this single chart is usually the most persuasive thing you show the jury.

10. Complete Development Order

PDF REQUIREMENT — this is the sequence the roadmap gives; I am not inventing a schedule, only restating it as phase $\rightarrow$ objective $\rightarrow$ deliverable $\rightarrow$ gate (already detailed per-phase in Section 4, summarized here for planning):

Phase	Objective	Definition of DONE (stop/go gate)
0	Freeze scope, pick simulator	Team agrees what “decentralized” means
1	Warehouse graph	Valid paths returned for any start/goal
2	One robot	20 repeated A $\rightarrow$ B runs, no bugs
3	A*	Correct least-cost routes on known maps
4	3 independent robots	Tasks complete with no central planner
5	Distributed auction	Lower insertion-cost robot wins bid tests
6	INTENT + reservation + arbitration	Zero collisions on repeated junction tests
7	Deadlock/single-lane proof	No cyclic wait, no head-on standoff
8	Traffic learning	Routes adapt to congestion; safety unaffected
9	Replanning/exceptions	Each failure type behaves as documented

10	Localization/clock	Stale beacon reduces throughput, not safety
11	Benchmark + baseline	Zero collisions, $\geq 20\%$ makespan gain, ≥ 10 seeds
12	Dashboard	Fleet keeps working with dashboard disconnected
13	Zoning (optional)	Auction traffic scoped to zone, not whole fleet
14	Physical demonstrator	Stable 1→2→3 robot hardware demo

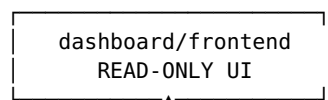
The PDF's one-line rule to keep visible: **"MAKE ONE ROBOT WORK → MAKE THREE ROBOTS COORDINATE → PROVE IT WITH DATA → THEN ADD HARDWARE."**

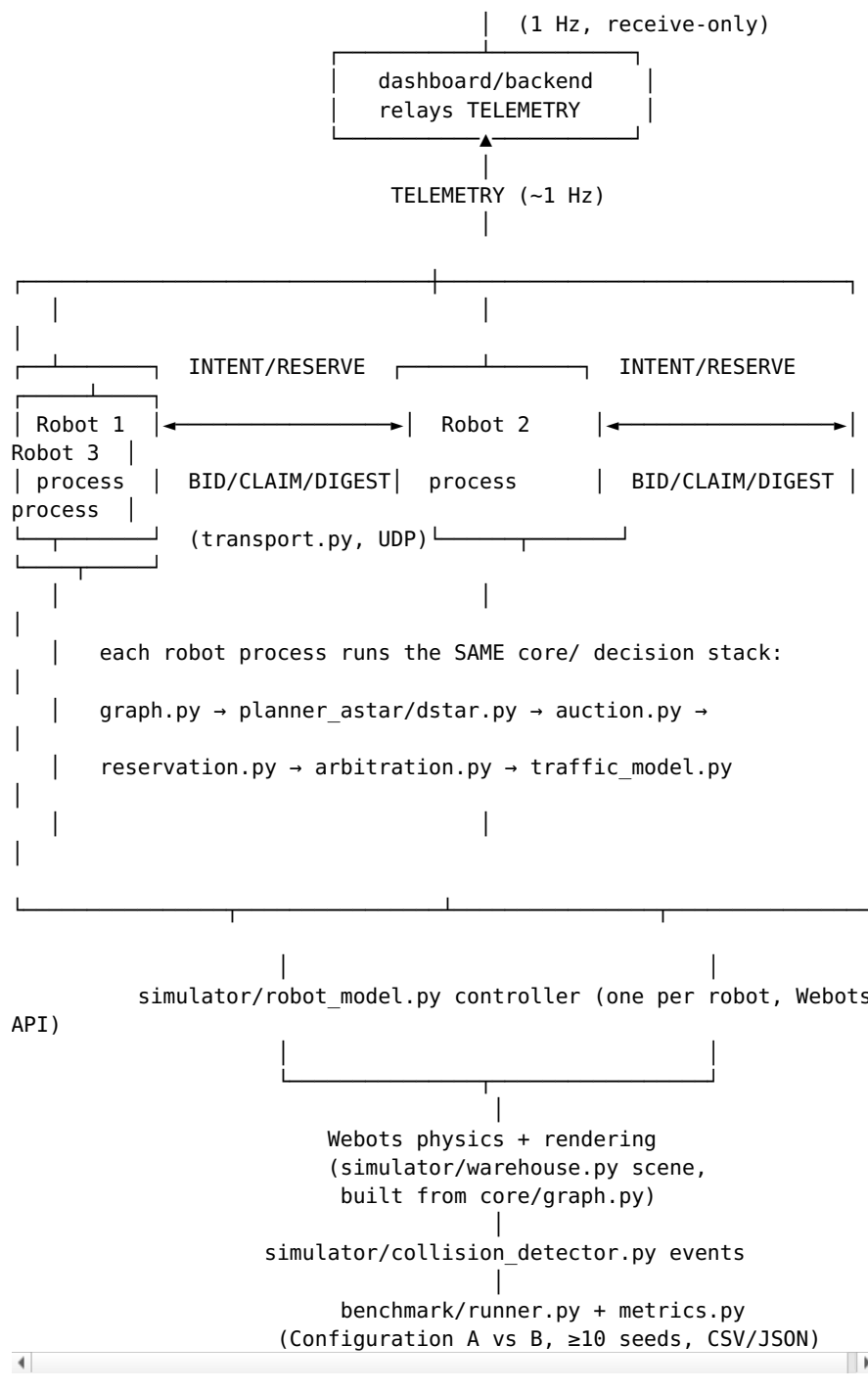
11. What We Should NOT Build (and why)

PDF REQUIREMENT — all six items below are explicitly listed as things to avoid for the first successful demo:

- **SLAM** — out of scope for release 1.0; the SRS assumes a *known* topological graph. Building SLAM burns weeks solving a problem the project doesn't need solved — use the fixed graph instead.
 - **Central task planner** — defeats the entire point of "decentralized"; if one server assigns routes, you have not built ROBOTON, you've built a conventional fleet manager. Use the distributed auction instead.
 - **Dashboard as a controller** — any "click to command a robot" button breaks the passive-monitoring requirement and undermines your AC-6-equivalent claim that the fleet survives gateway/dashboard loss. Keep it strictly receive-only.
 - **Learned traffic model inside safety arbitration** — if EWMA/pheromone values can affect *whether* a robot is allowed into a junction (not just which route it prefers), a desynchronized model between robots could cause a collision. Keep traffic learning purely as a route-cost input to *A/D*, never as an input to *arbitration.py*.
 - **100 physical robots** — hardware at that scale is a logistics and cost problem, not an algorithm problem; the roadmap explicitly says to demonstrate scale in *simulation* (Phase 13) and keep the physical demo to 2-3 robots.
 - **Excessive UI/CAD before algorithms work** — polishing the dashboard or 3D robot models before Phases 1-11 are solid wastes time on the part of the project that carries the least grading weight (P2 in the roadmap's priority table) while risking the P0 items (simulation, planner, auction, arbitration, benchmark) being unfinished.
-

12. Final "How Everything Connects" Diagram





13. Final Demo Flow (as given in the PDF)

PDF REQUIREMENT — this exact sequence: 1. Show the warehouse map, three robots receiving tasks. 2. Show distributed bids and the winning robot for a task. 3. Send two robots toward the same junction — show the reservation conflict live. 4. Show the higher-ranked robot reserving and the other yielding, zero collision. 5. Block an aisle — show route repair and peer awareness. 6. Disconnect the gateway — show held tasks continue. 7. Show the benchmark comparison: stop-and-wait vs. ROBOTON. 8. Show final metrics: makespan, collision count, stopped time, route diversity. 9. Finish with the physical 2-3 robot demonstrator, only if stable.

One-sentence story to tell the jury (PDF's own framing):

"Conventional coordination makes robots wait; ROBTON lets robots make local decisions, negotiate shared space, and adapt to congestion while keeping safety decisions deterministic."

Where to go from here

This document is the architecture map. The natural next step is to go phase-by-phase starting at **Phase 0/1** and write the actual `core/graph.py` + map visualizer together — that's the first thing with a testable, visible output.